



**SCHOOL OF BUSINESS AND MANAGEMENT SCIENCES
DEPARTMENT OF FINANCIAL ENGINEERING**

MITIGA GLOBAL ANALYTICS ENGINE

BY

**KUWANA TADIWANAI SHE J
H220006Z**

SUPERVISOR: MR MANGWANDA

**THIS PROTOTYPE IS SUBMITTED TO THE DEPARTMENT OF FINANCIAL
ENGINEERING, SCHOOL OF BUSINESS AND MANAGEMENT SCIENCES, IN
PARTIAL FULLFILLMENT OF THE BACHELOR OF TECHNOLOGY HONOURS
DEGREE IN FINANCIAL ENGINEERING**

2026

Title Page

Title of project: MITIGA GLOBAL ANALYTICS ENGINE

Author: KUWANA TADIWANAISHE J

University Name: HARARE INSTITUTE OF TECHNOLOGY

School Name: BUSINESS AND MANAGEMENT SCIENCES

Department Name: DEPARTMENT OF FINANCIAL ENGINEERING

Year: 2026

Declaration

I the author, KUWANA TADIWANASHE J H220006Z do hereby declare that this work has not previously been accepted in substance for any degree and is not being concurrently submitted in candidature for any degree.

Student Signature Date

Supervisor's Signature Date

Copyright

All rights reserved. No part of this research may be reproduced, stored in any retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise from scholarly purpose without the prior written permission of the author and Harare Institute of Technology on behalf of the author.

Dedication

This prototype is dedicated to my mother and father, who supported my education every step of the way. My brothers and friends kept me going when things got hard. This one is for all of you.

Acknowledgements

I would like to thank Mr Mangwanda for his guidance and steady support throughout this project. His feedback was always clear and practical. It made a real difference.

I also thank the research coordinators in the Department of Financial Engineering. Their structure and direction kept the project on track, especially in the later stages when things got complicated.

To my friends at HIT — four years went fast. The things we learned together, inside and outside the classroom, I will carry for a long time. Thank you.

Contents

Title Page	i
Declaration	ii
Copyright	iii
Dedication	iv
Acknowledgements	v
Abstract	x
Abbreviation	xi
Chapter 1: Introduction	1
1.1 Problem Statement	1
1.2 System Objectives	1
Chapter 2: Process Modelling	3
2.1 Introduction	3
2.2 Process Model (Iterative)	3
2.3 Feasibility Analysis	4
2.4 System Security	4
Chapter 3: Software Methodology	6
3.1 Introduction	6
3.2 System Flowchart	6
3.3 System Description	7
3.3.1 Step-1: Login	7
3.3.2 Landing Page	8
3.3.3 Admin Dashboard	9
3.3.4 Register Borrower	10
3.3.5 Borrower Detail and Field Analysis	11
3.3.6 Portfolio Analysis	12

3.3.7 Borrower Dashboard	13
Appendix: System Code	14

List of Figures

1	Process Model (Iterative)	3
2	System Flowchart	6
3	Login Page	7
4	Landing Page	8
5	Admin Dashboard	9
6	Register Borrower	10
7	Borrower Detail and Field Analysis	11
8	Portfolio Analysis	12
9	Borrower Dashboard	13

List of Tables

1	Glossary of Abbreviations	xi
---	-------------------------------------	----

Abstract

The Mitiga Global Analytics Engine is a web-based prototype that implements the research findings from the dissertation on physical climate risk in Zimbabwe's microfinance sector. The system brings the Two-Regime Markov-Switching credit migration model to life as a working application. Satellite data from Google Earth Engine is used to compute NDVI-based agricultural health scores and a field-level drought index (γ) for each registered borrower. This drought index drives the logistic regime weight $\omega(\gamma)$, which blends the normal and drought-stress migration matrices to produce a dynamic Probability of Default. From there, the system computes IFRS 9-compliant Expected Credit Loss across four forward-looking scenarios. Credit officers can manage the full borrower lifecycle through an admin dashboard, while farmers access a simplified view of their farm health and loan status. The prototype demonstrates that the theoretical framework is not only mathematically valid but also operationally feasible.

Abbreviation

Abbreviation	Full Form
IFRS 9	International Financial Reporting Standard 9
ECL	Expected Credit Loss
PD	Probability of Default
LGD	Loss Given Default
EAD	Exposure at Default
SPEI	Standardized Precipitation Evapotranspiration Index
NDVI	Normalized Difference Vegetation Index
GEE	Google Earth Engine
MFI	Microfinance Institution
API	Application Programming Interface
UI	User Interface
TTC	Through-the-Cycle
PIT	Point-in-Time

Table 1: Glossary of Abbreviations

Chapter 1

INTRODUCTION

The research dissertation validated the Two-Regime Markov-Switching model as a rigorous approach to estimating Expected Credit Loss under physical climate risk. Chapter 4 of the study showed that conditioning credit migration on the drought index produces materially different default probabilities during stress periods, with the AUC improving from 0.618 under the static single-matrix model to 0.724 under the two-regime model. The Mitiga Global Analytics Engine takes that validated framework and turns it into a working web application. Credit officers can register borrowers, map their farmland, retrieve satellite NDVI data from Google Earth Engine, and compute a field-level drought gamma for each farm. The system then applies the regime-switching logic to estimate ECL under four IFRS 9-compliant forward-looking scenarios.

1.1 Problem Statement

Microfinance Institutions in Zimbabwe face a very specific credit risk problem. Most of their borrowers are smallholder farmers. When drought hits, repayment capacity drops sharply and it does not happen gradually — it happens all at once, across the whole portfolio. Static credit risk models miss this completely. They estimate an average probability of default that stays the same regardless of whether the region is wet or dry. The result is chronic under-provisioning during exactly the periods when losses are highest.

The research quantified this gap. Portfolio ECL under the two-regime model was significantly higher during drought conditions than the figure produced by the static model. That difference represents real capital exposure that MFIs are currently ignoring. Without a tool that implements the regime-switching logic, credit officers have no way to convert a satellite drought reading into an IFRS 9 loss estimate. The Mitiga Global Analytics Engine was built to close that gap.

1.2 System Objectives

The prototype was built to meet the following specific objectives:

- To allow credit officers to register and manage microfinance borrowers, linking each to their registered farm field

- To use Google Earth Engine to fetch NDVI and weather data for drawn farm polygons, and to compute a field-level agricultural health score (0–100)
- To derive a drought index γ from the health score and apply the logistic regime weight $\omega(\gamma)$ to blend M_{normal} and M_{stress} migration matrices
- To compute 1-year and lifetime ECL under four IFRS 9 scenarios: baseline, moderate drought, severe drought, and wet recovery
- To aggregate individual borrower risk into a portfolio view showing regime distribution and scenario-weighted ECL
- To provide borrowers with a plain-language dashboard showing their farm health score and loan risk status

Conclusion

The Mitiga Global Analytics Engine makes the research framework operational. Every step in the system — from the GEE satellite fetch to the final ECL figure — maps directly to equations and parameters from the dissertation. The chapters that follow explain how the system was designed, what process model was used, and how each screen in the application works.

Chapter 2

PROCESS MODELLING

2.1 Introduction

The system was built using an iterative prototyping approach. This was a deliberate decision. The Two-Regime Markov-Switching model at the core of the application has several interacting parameters — the regime sharpness κ , the drought threshold γ_0 , the LGD assumption, and the structure of the two migration matrices. Getting those parameters to produce sensible outputs required building early versions and testing them against known results from the dissertation. A waterfall approach would have locked the design in too early. Iteration was the only practical path.

2.2 Process Model (Iterative)

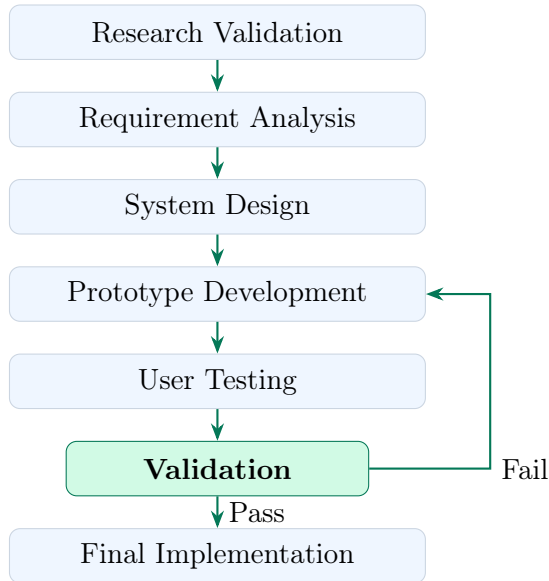


Figure 1: Process Model (Iterative)

The iterative model was the right fit for three reasons. First, the regime parameters needed real calibration cycles. The threshold value $\gamma_0 = -1.10$ was arrived at after testing several values against the dissertation’s empirical data. Getting it wrong early would have produced wrong ECL numbers throughout. Second, the Next.js web stack supported fast iteration — a change to the API or a UI fix was live in

seconds, which kept development moving. Third, the borrower registration and field analysis workflow went through several revisions based on test runs. Each time it became simpler. That kind of refinement only happens when you build, test, and build again.

2.3 Feasibility Analysis

- **Economic Feasibility:** The entire system uses open-source technologies. Next.js, Prisma, SQLite, and TypeScript are all free. Google Earth Engine provides satellite data at no cost for academic use. There are no licensing fees anywhere in the stack.
- **Technical Feasibility:** The regime-switching computation is not expensive. The logistic weight $\omega(\gamma)$, the matrix blending, and the ECL summation all run in milliseconds inside Next.js API routes. There is no need for a dedicated compute server. A standard laptop or small cloud instance is sufficient.
- **Social Feasibility:** The system serves two very different users. Credit officers are comfortable with tables and numbers — the admin dashboard reflects that. Borrowers are farmers who may not be comfortable with financial terminology — the borrower dashboard is intentionally stripped of technical language. Both groups were considered separately from the start.
- **Operational Feasibility:** The system is a web application. Any device with a browser can use it. No installation is required on the user side. The SQLite database keeps the backend simple with no external database server to configure.

2.4 System Security

The system has two layers of security. The first is role-based access control. There is no public signup for credit officers — admin accounts are created directly by the system developer. Borrowers can register through a public page but can only see their own data once logged in. The second layer is password security. All passwords are stored as bcrypt hashes. Sessions use HTTP-only cookies which cannot be read by client-side JavaScript. Every protected page checks the session and user role on the server side before rendering anything.

Conclusion

Iterative prototyping gave the flexibility needed to calibrate model parameters and refine the user interface through repeated cycles of building and testing. The feasibility study shows the system is economically and technically practical. Security is handled at the authentication layer with role separation and secure session management. The next chapter covers the actual application screens and how they implement the research framework.

Chapter 3

SOFTWARE METHODOLOGY

3.1 Introduction

This chapter describes the system from the user's side. Each main screen is presented and explained. The Mitiga Global Analytics Engine is a Next.js web application backed by a Prisma ORM and a SQLite database. It has two user roles: Administrator (the credit officer) and Borrower (the farmer). Each role sees a different part of the system. Wireframes and screen descriptions are given below.

3.2 System Flowchart

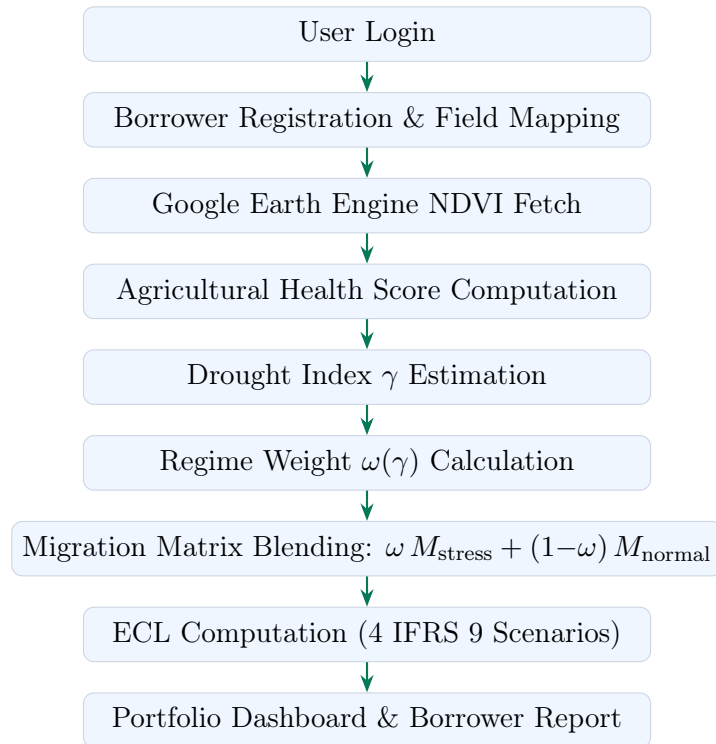


Figure 2: System Flowchart

The flowchart traces the data from login all the way to the portfolio report. The Google Earth Engine step is the critical one. It pulls NDVI and 14-day weather data for the farm polygon drawn by the admin. That raw satellite data feeds the agricultural health score. The score feeds the drought gamma. Gamma drives the regime weight. The weight determines how much of the stress matrix versus the

normal matrix goes into the final migration probabilities. Everything downstream depends on that chain.

3.3 System Description

3.3.1 Step-1: Login

Mitiga Global Analytics Engine

IFRS 9 Physical Climate Risk Platform

- ✓ IFRS 9 compliant ECL framework
- ✓ Four-scenario drought stress testing
- ✓ SPEI-12 satellite drought index

Research prototype · 2026

Welcome back
Sign in to your Mitiga account

Email
you@example.com

Password
••••••••

Sign In

No account? [Contact your credit officer](#)

Figure 3: Login Page

The login page is the entry point for all users. It accepts an email address and password. After successful login, the system reads the user's role from the database. Administrators are redirected to the admin dashboard. Borrowers go to their personal dashboard. The left panel of the page shows the system name and three feature highlights: IFRS 9 compliance, four-scenario drought stress testing, and the SPEI-12 satellite drought index. This gives a visitor a clear picture of what the system does before they even sign in.

3.3.2 Landing Page

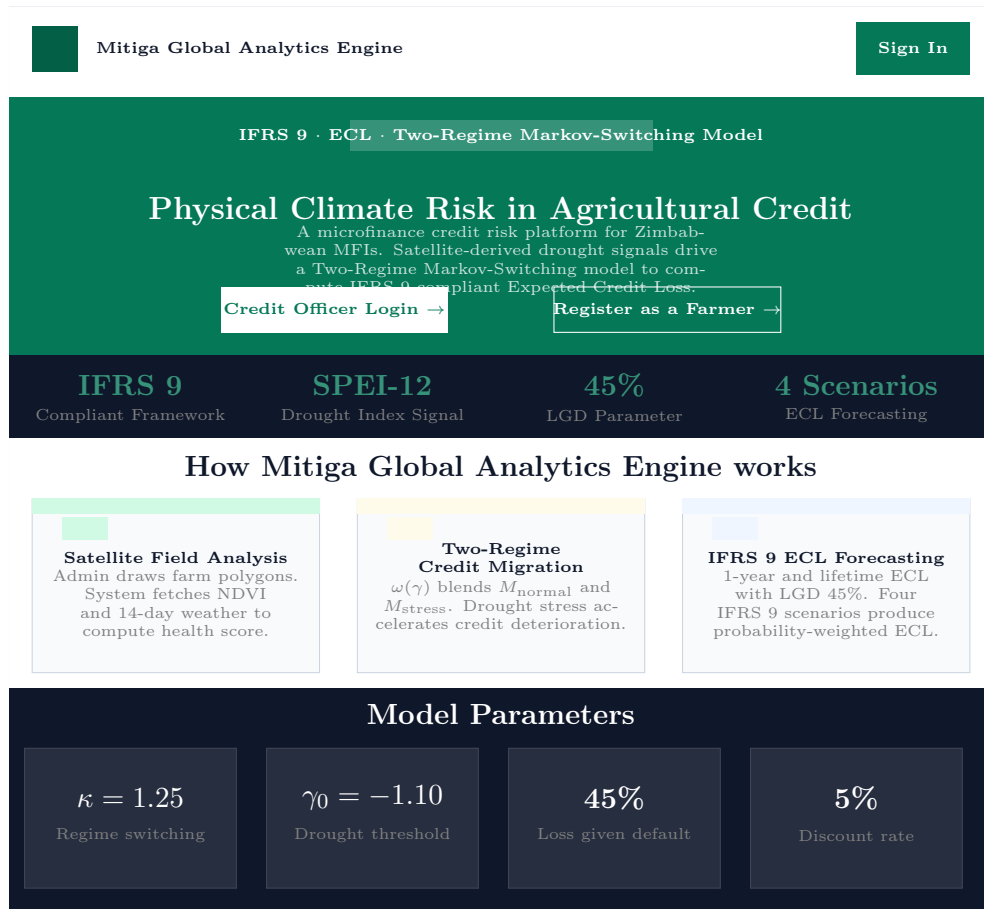


Figure 4: Landing Page

The landing page is publicly accessible. It explains the system at a high level. The hero section at the top has two buttons: one for credit officer login and one for farmer self-registration. Below that, three cards summarise the three main functions of the system. A statistics strip shows four headline metrics. The dark parameter section at the bottom makes it clear that this is not a generic risk tool — it implements specific, calibrated values from the research.

3.3.3 Admin Dashboard

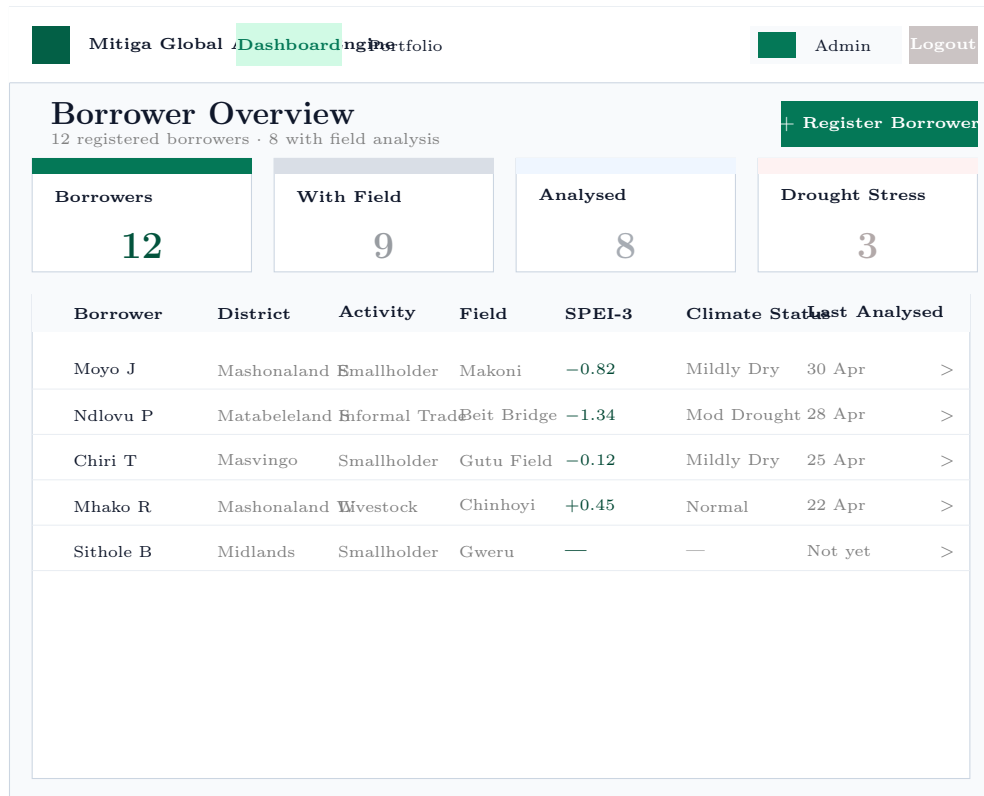


Figure 5: Admin Dashboard

The admin dashboard is the main working screen for credit officers. The four summary cards at the top give a quick overview of the portfolio state. The drought stress card is highlighted in red and shows the count of borrowers whose SPEI gamma is below -1.0 — the threshold where the research shows default probabilities increase sharply. The borrower table below shows the full list. Each row has the SPEI-3 gamma value and a climate status label. Clicking the arrow on any row opens the detailed borrower profile.

3.3.4 Register Borrower

Mitiga Global Analytics Engine Logout

[← Back](#)
Register Borrower
Link a self-registered farmer to a borrower profile

Select Farmer

Farmer Account
Select a registered farmer... ▾

Phone Number
+263...

National ID
e.g. 63-2345678A21

District
Mashonaland East ▾

Primary Activity
Smallholder Agriculture ▾

Register Borrower

Loan details and field mapping are added on the borrower detail page after registration.

Figure 6: Register Borrower

New borrowers are added to the portfolio by the admin. The registration page shows a dropdown of farmers who have signed up through the public registration page. The admin selects the farmer and fills in the remaining profile details: phone number, national ID, district, and primary economic activity. This two-step design means basic account creation is open to farmers, but sensitive borrower profiling and loan data stays under admin control. The district dropdown covers all eight Zimbabwean provinces that fall within the research’s geographic scope.

3.3.5 Borrower Detail and Field Analysis

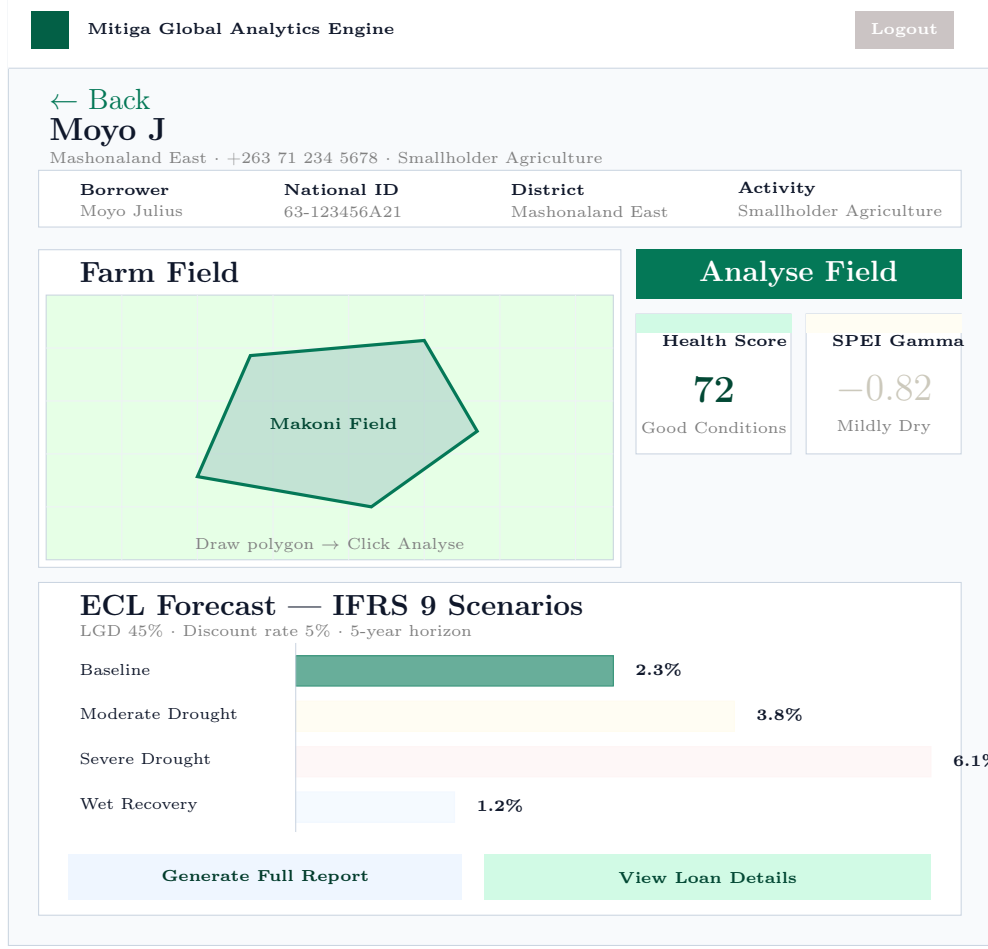


Figure 7: Borrower Detail and Field Analysis

The borrower detail page is where the actual climate risk analysis takes place. The admin draws a polygon on the embedded map to mark the farm boundaries. Clicking Analyse sends the polygon coordinates to the backend, which forwards them to Google Earth Engine. GEE returns NDVI and weather data for that location. The system computes the agricultural health score from five inputs: mean NDVI, NDVI trend, average temperature, total rainfall, and water stress risk. The score maps to a drought gamma value. Gamma determines the regime weight $\omega(\gamma)$, which blends M_{normal} and M_{stress} to produce the climate-adjusted PD. ECL is then computed as:

$$\text{ECL} = \sum_{t=1}^T \frac{\text{PD}_t \times \text{LGD} \times \text{EAD}}{(1+r)^t}$$

with LGD fixed at 45% and a 5% annual discount rate, matching the dissertation's calibration.

3.3.6 Portfolio Analysis

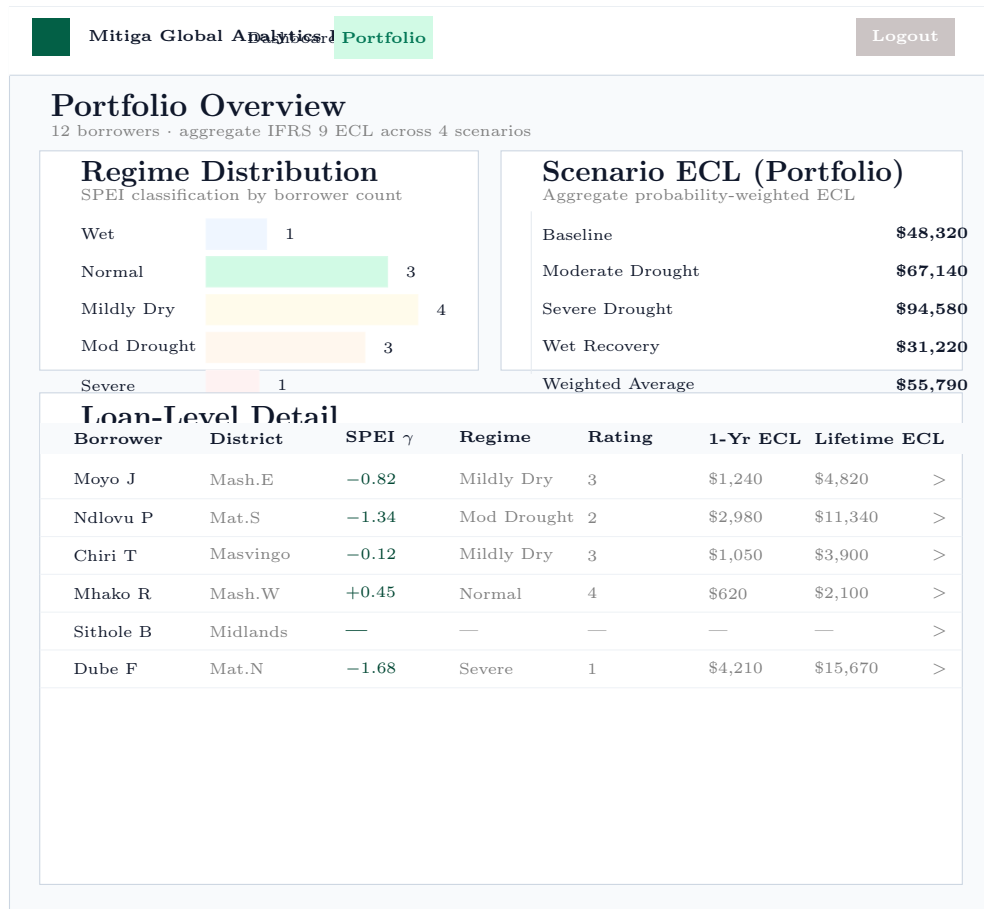


Figure 8: Portfolio Analysis

The portfolio page aggregates all borrower results into a single view. The regime distribution bars on the left show immediately how much of the portfolio is sitting in drought-stress territory. This is the key early-warning signal for MFI management. If a large fraction of the portfolio is in the moderate or severe drought regime, ECL provisioning needs to go up. The scenario ECL summary on the right shows four aggregate ECL figures for the whole portfolio. The loan table at the bottom gives per-borrower detail so the credit officer can identify the specific accounts driving the highest ECL figures.

3.3.7 Borrower Dashboard

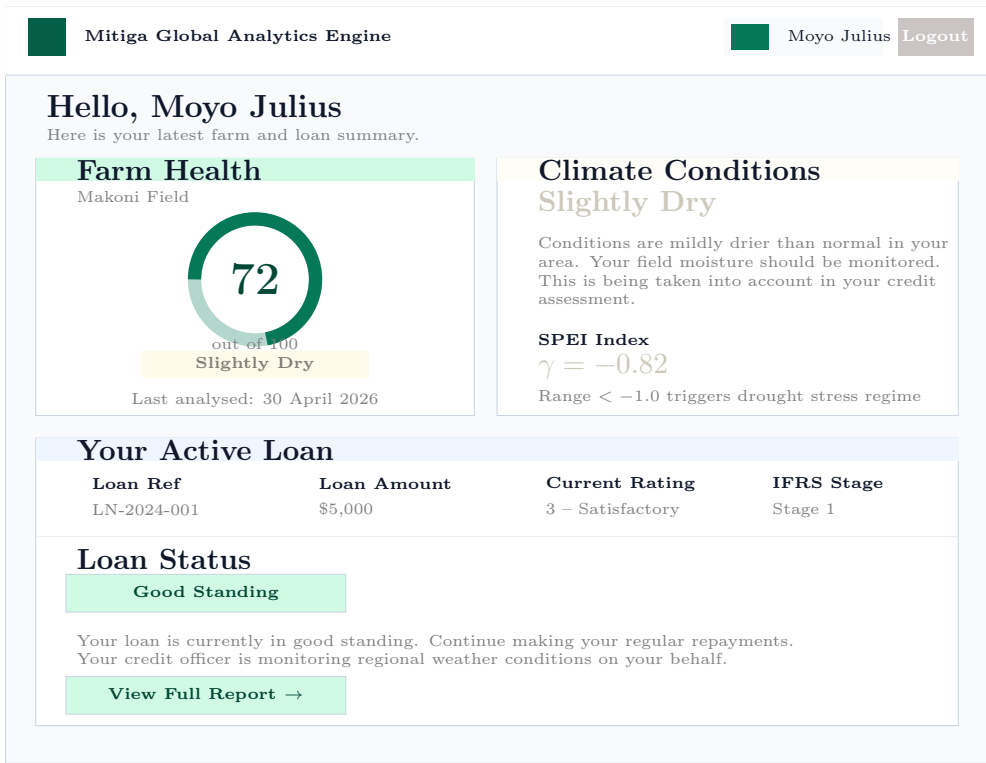


Figure 9: Borrower Dashboard

The borrower dashboard is designed for farmers, not analysts. It shows none of the model parameters. The farmer sees their farm name, an agricultural health score displayed inside a progress ring, a colour-coded status badge, and a plain-language description of current climate conditions. The loan card below uses simple language to describe the loan status. If conditions worsen, the status badge changes from green to amber to red and the explanatory text updates accordingly. The goal is to keep the farmer informed without overwhelming them with financial terminology.

Conclusion

The implemented system faithfully translates the research framework into a working application. Each screen has a clear purpose and serves either the credit officer or the farmer. The flowchart in Section 3.2 maps directly to the screens above. From the GEE satellite fetch through to the portfolio ECL summary, every step is traceable back to the Two-Regime Markov-Switching model and the IFRS 9 ECL formula from the dissertation.

APPENDIX

System Code

The following Python scripts form the machine learning pipeline that trains the agricultural health model used inside the Mitiga Global Analytics Engine. The trained model is exported as a JSON file containing the logistic regression coefficients and intercept, which the Next.js server reads at runtime to classify field health without calling an external Python service.

config.py

```
1 import os
2 from pathlib import Path
3
4 # Base Paths
5 BASE_DIR = Path(__file__).resolve().parent.parent
6 DATA_DIR = BASE_DIR / "data"
7 RAW_DATA_PATH = DATA_DIR / "raw" / "dataset.csv"
8 PROCESSED_DATA_DIR = DATA_DIR / "processed"
9 MODEL_DIR = BASE_DIR / "models"
10 REPORT_DIR = BASE_DIR / "reports"
11
12 # Ensure directories exist
13 PROCESSED_DATA_DIR.mkdir(parents=True, exist_ok=True)
14 MODEL_DIR.mkdir(parents=True, exist_ok=True)
15 REPORT_DIR.mkdir(parents=True, exist_ok=True)
16
17 # Feature Configuration
18 FEATURES = [
19     "mean_ndvi",
20     "ndvi_trend",
21     "ndvi_variance",
22     "avg_temperature_c",
23     "total_rainfall_mm"
24 ]
25 TARGET = "crop_health_label"
26
27 # Model Export Path
28 MODEL_EXPORT_PATH = MODEL_DIR / "crop_health_model.json"
29 THRESHOLDS_PATH = MODEL_DIR / "thresholds.json"
```

Listing 1: config.py — Project paths and feature configuration

feature_engineering.py

```

1 """
2 Feature engineering logic.
3 In a production app, this would process raw GEE output into
4 model-ready features such as mean NDVI and rainfall totals.
5 For this prototype, dataset.csv already contains these features.
6 """
7
8 def process_features(raw_data):
9     # Placeholder for logic that converts raw GEE output to model
10    features
11    return raw_data

```

Listing 2: feature_engineering.py — Feature processing logic

labeling.py

```

1 """
2 Labeling logic for training data.
3 Converts NDVI and weather patterns into 'HEALTHY',
4 'MODERATE_STRESS', or 'HIGH_STRESS'.
5 """
6
7 def apply_labels(df):
8     # Rule-based labeling used to generate training data
9     # if labels were not already present in the dataset.
10    return df

```

Listing 3: labeling.py — Rule-based training label generation

train_health_model.py

```

1 import pandas as pd
2 import joblib
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.metrics import classification_report,
6     confusion_matrix
7 import matplotlib.pyplot as plt
8 import seaborn as sns
9 from config import RAW_DATA_PATH, FEATURES, TARGET, MODEL_DIR,
10     REPORT_DIR
11
12 def train_model():
13     print(f>Loading data from {RAW_DATA_PATH}...)
14     df = pd.read_csv(RAW_DATA_PATH)

```

```

14 X = df[FEATURES]
15 y = df[TARGET]
16
17 X_train, X_test, y_train, y_test = train_test_split(
18     X, y, test_size=0.2, random_state=42
19 )
20
21 print("Training Logistic Regression Classifier...")
22 model = LogisticRegression(max_iter=1000, random_state=42)
23 model.fit(X_train, y_train)
24
25 # Evaluation
26 y_pred = model.predict(X_test)
27 report = classification_report(y_test, y_pred)
28 print("\nClassification Report:")
29 print(report)
30
31 # Save report
32 with open(REPORT_DIR / "training_summary.md", "w") as f:
33     f.write("# Training Summary\n\n")
34     f.write("## Classification Report\n")
35     f.write("```\n")
36     f.write(report)
37     f.write("```\n")
38
39 # Confusion Matrix Plot
40 plt.figure(figsize=(8, 6))
41 cm = confusion_matrix(y_test, y_pred)
42 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
43             xticklabels=model.classes_,
44             yticklabels=model.classes_)
45 plt.title('Confusion Matrix')
46 plt.ylabel('Actual')
47 plt.xlabel('Predicted')
48 plt.savefig(REPORT_DIR / "confusion_matrix.png")
49
50 # Save model (joblib for Python; JSON export handled
51 separately)
52 joblib.dump(model, MODEL_DIR / "crop_health_model.joblib")
53 print(f"Model saved to {MODEL_DIR / 'crop_health_model.joblib'
54       '}")
55
56 return model
57
58 if __name__ == "__main__":
59     train_model()

```

Listing 4: train_health_model.py — Logistic Regression training script

export_model.py

```
1 import json
2 import joblib
3 import numpy as np
4 from config import MODEL_DIR, FEATURES, MODEL_EXPORT_PATH,
   THRESHOLDS_PATH
5
6 def export_model():
7     model_path = MODEL_DIR / "crop_health_model.joblib"
8     if not model_path.exists():
9         print("Model file not found. Please train the model first.")
10    return
11
12    print(f"Loading model from {model_path}...")
13    model = joblib.load(model_path)
14
15    # Export Logistic Regression coefficients and intercept
16    model_data = {
17        "model_type": "LogisticRegression",
18        "features": FEATURES,
19        "classes": model.classes_.tolist(),
20        "coefficients": model.coef_.tolist(),
21        "intercept": model.intercept_.tolist()
22    }
23
24    with open(MODEL_EXPORT_PATH, "w") as f:
25        json.dump(model_data, f, indent=2)
26
27    print(f"Model exported to {MODEL_EXPORT_PATH}")
28
29    # Export classification thresholds
30    thresholds = {
31        "ndvi": {
32            "healthy": 0.6,
33            "moderate": 0.45
34        },
35        "rainfall": {
36            "low": 20
37        },
38        "temperature": {
39            "high": 30
```

```

40     }
41 }
42
43 with open(THRESHOLDS_PATH, "w") as f:
44     json.dump(thresholds, f, indent=2)
45
46     print(f"Thresholds exported to {THRESHOLDS_PATH}")
47
48 if __name__ == "__main__":
49     export_model()

```

Listing 5: `export_model.py` — Exports trained model coefficients to JSON for the Next.js application